

SmartHire

Resume-to-Job Matching & Career Guidance Engine — Final Project Report

1. Project Overview

SmartHire is a classical machine learning system that lets a candidate upload a resume and receive: (a) a predicted career category, (b) a ranked list of matching jobs drawn from a merged job corpus, (c) a fit/shortlisting score for each recommended job, and (d) a skill-gap report comparing the candidate's vocabulary against their best-matching jobs. No large language models or generative AI are used at any stage; every prediction is produced by TF-IDF vectorization, cosine similarity, Logistic Regression / Random Forest / Gradient Boosting, and K-Means clustering.

2. Data

Three public datasets were combined:

- Resume dataset — 2,679 labeled resumes across 25 job categories, used to train the classifier.
- Naukri job postings — Indian job listings with title, company, skills, experience, and description.
- LinkedIn job postings (2023–2024) — a larger set of detailed job descriptions, used to broaden job variety.

The Naukri and LinkedIn datasets were standardized into a common schema (title, company, location, skills, description, experience, source) and concatenated into a single job corpus of 61,019 postings (36,643 from Naukri, 24,376 from LinkedIn).

2.1 Data Cleaning

- Resume text had a double-encoding corruption (UTF-8 stored as Latin-1) that was reversed before use.
- Duplicate resumes were dropped (2,679 → 2,461 rows).
- Three category labels — "Java Developer", "DevOps Engineer", and "Testing" — were removed after EDA revealed inconsistent or overlapping content under these labels, leaving 22 usable categories for the final classifier.
- All text (resumes and job postings) was lowercased, stripped of punctuation, and had whitespace collapsed before vectorization.

3. Supervised Component: Resume Category Classifier

Resume text was vectorized with TF-IDF (5,000 features) and used to train two classifiers on an 80/20 train-test split (1,968 train / 492 test rows):

- Logistic Regression — baseline linear classifier.
- Random Forest (200 trees) — final chosen model, evaluated after removing the three problematic categories.

Results:

- Random Forest accuracy: 0.70 (vs. 0.66 for Logistic Regression).
- Random Forest ROC-AUC (macro, one-vs-rest): 0.97.

The gap between accuracy (0.70) and macro ROC-AUC (0.97) is expected in a 22-class problem: ROC-AUC in a one-vs-rest setting rewards a model for correctly ranking each class against the rest even when the single top prediction is wrong, so it is far less punishing than top-1 accuracy on a task this fine-grained. Per-class performance was uneven — well-represented, distinctive categories (e.g. Accountant) scored precision/recall around 0.80, while categories with smaller or more generic vocabularies (e.g. Agriculture) scored substantially lower, most visibly in the confusion matrix (see reports/figures/confusion_matrix_logreg.png).

4. Unsupervised Component: Job Recommender

Job descriptions and resumes were vectorized with a shared TF-IDF space (10,000 features), and recommendations are produced by ranking jobs on cosine similarity to the resume vector, returning the top-N matches with a match score.

Evaluation used a proxy Precision@K: since no dataset exists pairing a resume with a single 'correct' job, a recommended job is counted as a 'hit' if its title shares a category-related keyword with the resume's known label (e.g. a HR resume matching a job titled 'HR Manager'). Under this proxy, Precision@10 was 0.25 averaged over 50 sampled resumes. Manual spot-checks (e.g. for Chef and Aviation resumes) confirmed the recommender surfaces semantically relevant roles even when exact keyword overlap is imperfect, and a check across all 25 categories found no resumes with an outright label/content mismatch.

5. Unsupervised Component: Job Clustering & Skill-Gap Report

K-Means was run on the job TF-IDF vectors for $k = 4, 6, 8, 10, 12$, using the elbow method (inertia) and silhouette score to guide the choice of k . Silhouette scores were low across all tested values (0.020–0.027), which is common for short, keyword-heavy job postings in high-dimensional sparse TF-IDF space — the ten clusters found have overlapping vocabularies rather than sharp, well-separated boundaries. $k=10$ was selected as a practical balance between inertia gains and interpretability of per-cluster top keywords (e.g. a clear customer-support/BPO cluster centered on 'voice, process, bpo, inbound, chat').

The skill-gap module compares a resume's vocabulary against the top TF-IDF terms of its most similar jobs (after filtering out generic boilerplate words like 'experience', 'team', and 'position'), reporting which of those terms the candidate already shows and which are missing. This module was iterated through several versions during development to fix an early bug where the 'skills' field was frequently empty for Naukri postings, which was resolved by falling back to keyword extraction from the full job description text.

6. Supervised Component: Fit / Shortlisting Predictor

A binary fit predictor was added to estimate how well a candidate matches a specific job, going beyond plain similarity ranking. Since no dataset exists that records real shortlisting outcomes, training pairs were generated with weak supervision: for a sample of 300 resumes, each resume's top-3 most similar jobs (by TF-IDF cosine similarity) were labeled as positive ('good fit') and 3 jobs drawn from the bottom half of its similarity ranking were labeled as negative ('poor fit').

Feature engineering (`src/features/match_features.py`) combined the text similarity score with structured signals: skill-keyword overlap count and ratio between resume and job skills, years-of-experience extraction from free text compared against the job's stated experience range, an experience-match flag and gap, an education-keyword overlap flag, and resume length.

Two models were trained and compared on an 80/20 stratified split: Logistic Regression (with class balancing) and Gradient Boosting. Both achieved very high accuracy and ROC-AUC (see Results Summary below). This is expected rather than surprising: because positive pairs were defined as high-similarity and negative pairs as low-similarity, the `text_similarity` feature alone is a near-perfect separator of the two classes by construction. The model therefore demonstrates that the feature pipeline is technically sound and learns a sensible decision boundary, but its near-1.0 metrics reflect the synthetic labeling scheme rather than a validated ability to distinguish genuinely strong candidates from weak ones on real shortlisting decisions. This caveat is treated as a first-class limitation (Section 9), and the fit score is surfaced in the app as a directional signal alongside — not a replacement for — the match score.

The better-performing model (Gradient Boosting, by ROC-AUC) was saved as `models/fit_predictor.pkl` and integrated into the Streamlit app, where it scores every recommended job for the uploaded resume and displays the result as an additional 'Fit Score' column next to the TF-IDF match score.

7. System Architecture

Resume upload (PDF / DOCX / TXT) → text extraction (src/parsing/resume_parser.py) → shared text cleaning → four parallel branches: (1) TF-IDF + classifier → predicted category, (2) TF-IDF + cosine similarity → ranked jobs, (3) ranked jobs + engineered features → fit predictor → fit score per job, (4) similarity results → skill-gap report. All outputs are rendered in a single Streamlit page (app/streamlit_app.py), with model artifacts cached across reruns using `@st.cache_resource`.

8. Results Summary

Component	Metric	Value
Resume Classifier (Random Forest)	Accuracy	0.70
Resume Classifier (Random Forest)	ROC-AUC (macro, OvR)	0.97
Resume Classifier (Logistic Regression)	Accuracy	0.66
Job Recommender	Precision@10 (proxy)	0.25
Job Clustering (K-Means, k=10)	Silhouette Score	0.024
Fit Predictor (Logistic Regression)	Accuracy / ROC-AUC	0.997 / 0.999
Fit Predictor (Gradient Boosting)	Accuracy / ROC-AUC	0.997 / 1.000

9. Limitations

- The Fit Predictor's training labels are weak/synthetic (top-similarity pairs = positive, bottom-similarity pairs = negative), not real shortlisting outcomes. Its near-perfect accuracy and ROC-AUC reflect that the `text_similarity` feature makes the synthetic classes almost trivially separable, not a validated real-world shortlisting skill. The fit score should be read as a directional signal, not a certified probability of being hired or shortlisted.
- Precision@K for the recommender is a keyword-overlap proxy, not ground-truth precision, since no dataset pairs resumes with a single correct job.
- Clustering silhouette scores are low, meaning cluster boundaries are soft; cluster keyword summaries are directionally useful for the skill-gap module but should not be read as hard job-family divisions.
- Dropping three category labels (Java Developer, DevOps Engineer, Testing) improves reported classifier accuracy but slightly narrows the categories the classifier can predict; this trade-off was made for data quality reasons found during EDA.
- The Salary Band Predictor and topic modeling (LDA/NMF), both suggested as optional stretch goals, were not attempted.

10. Conclusion

SmartHire implements the full required pipeline plus one optional component — a supervised resume classifier, an unsupervised content-based job recommender, a supervised fit predictor, and a clustering-driven skill-gap module — integrated into a working Streamlit demo, entirely with classical machine learning techniques. The system demonstrates reasonable classification accuracy (0.70) with strong class-separation ability (0.97 macro ROC-AUC), a functioning content-based recommender validated against category-level proxies, an interpretable skill-gap report, and a fit-scoring layer whose construction and limitations are documented transparently given the absence of ground-truth resume-to-job outcome labels.